

RDF Query Language: Design and Implementation

Programming Language Concepts coursework report

1. Introduction

This report covers the language we designed for the RDF querying coursework, and the Haskell interpreter that runs it. The interpreter uses Alex for the lexer and Happy for the parser. A program loads one or more Turtle files, runs some combination of graph operations and pattern-matching queries on them, and prints the result as canonical N-Triples.

At the top level the language is imperative: you assign expressions to names and eventually print one of them. Inside an expression though, things look more like SQL or SPARQL: you say what triples you want, not how to get them. We found this split worked well for the tasks, since most of them are easier to write as a couple of named intermediate queries combined with set operations than as one giant query.

2. Why the language looks the way it does

2.1 SPARQL and SQL

The two obvious starting points for a language that queries RDF are SPARQL [1] (designed for exactly that) and SQL (which everyone has seen). We borrowed from both. SPARQL gave us the `?var` syntax for query variables, and the general shape of `SELECT ( . . . ) WHERE . . .`. SQL gave us `GROUP BY` and the convention of uppercasing keywords.

We did not try to be a full SPARQL implementation. SPARQL has property paths, optional patterns, several different query forms, and so on, none of which we needed for the ten tasks. We picked the smallest subset of features that would actually let us write the task programs cleanly, and tried to make the syntax more uniform than SPARQL's where we could. For example, every place a triple pattern appears in our language it has the same shape: `MATCH(s, p, o) IN g`. SPARQL puts patterns inside braces and uses position to distinguish parts of the query, which we found harder to read.

2.2 Naming intermediate results

An important early decision was that programs would be sequences of named assignments rather than a single big query. Task 7 is the clearest example of why. It asks for triples that match exactly one of three conditions, which is an exclusive-or. There is no nice way to write XOR as one boolean expression without either adding an XOR operator or doing a lot of negation. With named intermediates and the MINUS operator, we just compute each of the three sets, subtract the others from each one, and union the results. It is more lines of code but it is much easier to read and debug.

The same idea shows up in Tasks 3 and 10, where the answer is the union of two pattern matches. Rather than writing one query with an OR inside, we write two queries and union them. The semantics are the same and each piece is simpler.

3. Syntax

3.1 Tokens

The lexer recognises keywords (uppercase, reserved), identifiers (lowercase alphanumerics, used for naming graphs), query variables (a ? followed by an identifier), URIs (in angle brackets), strings (in double quotes), and unsigned integers. Whitespace is ignored. Line comments start with - - and run to the end of the line.

```
graph1 <- LOAD "foo"           -- load foo.ttl
result <- graph1 UNION graph2  -- combine, dedupe
PRINT result                   -- output as N-Triples
```

3.2 Programs

A program is one or more statements. There are only two kinds of statement: assignment, and print.

```
Statement ::= ident '<-' Expr
           | PRINT ident
```

We did not bother with a statement separator. Every statement starts with either an identifier or the PRINT keyword, and the lexer skips whitespace, so the parser can tell where one ends and the next begins.

3.3 Expressions

There are six expression forms:

```
Expr ::= LOAD string
       | ident UNION ident
       | ident INTERSECT ident
       | ident MINUS ident
       | SELECT '(' OutputList ')' WHERE Condition
       | SELECT '(' OutputList ')' WHERE Condition GROUP BY var
```

Set operations only take the names of existing graphs as operands, not nested expressions. This was a conscious choice to keep the grammar small and force the programmer to name intermediate results, which we think makes programs easier to follow.

3.4 SELECT

SELECT takes a list of three output terms (one each for the subject, predicate and object positions of the output triples) and a WHERE condition. The condition is built from MATCH patterns, comparisons, and the boolean operators AND, OR, NOT:

```
result <- SELECT (?s, <http://example.org/ont/hasAge>, ?o)
           WHERE MATCH (?s, <http://example.org/ont/hasAge>, ?o) IN baz
           AND ?o >= 21
```

MATCH(s, p, o) IN g goes through every triple in g and tries to unify each component with the corresponding term. A query variable binds to whatever it sees; a constant (URI, string or integer) only matches itself. Comparisons (=, !=, >=, <=, >, <) filter bindings.

If you put an aggregate function in the output list, you also need a GROUP BY clause, like in SQL:

```
result <- SELECT (?x, <http://example.org/ont/price>, MAX(?p))
           WHERE MATCH(?x, <http://example.org/ont/price>, ?p) IN quux
           GROUP BY ?x
```

3.5 Two worked examples

Task 5 is a good example of how joins work. It asks: if *x* is of type *y* and *y* is a subclass of *z*, output that *x* is also of type *z*. The two facts come from different graphs:

```
xyzyy <- LOAD "xyzyy"
plugh <- LOAD "plugh"

result <- SELECT (?x, <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>, ?z)
  WHERE MATCH (?x, <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>, ?y) IN xyzyy
  AND MATCH (?y, <http://www.w3.org/2000/01/rdf-schema#subClassOf>, ?z) IN plugh

PRINT result
```

The two **MATCH** patterns share *?y*, which forces a join: a binding only counts if both patterns can be satisfied with the same value of *?y*. The output uses *?z* in the object position to produce the new triple.

Task 8 is the negation example. It asks for `foaf:Person` resources who do *not* know `nmg`:

```
foo <- LOAD "foo"
result <- SELECT (?x, <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>,
  <http://xmlns.com/foaf/0.1/Person>)
  WHERE MATCH (?x, <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>,
    <http://xmlns.com/foaf/0.1/Person>) IN foo
  AND NOT MATCH (?x, <http://xmlns.com/foaf/0.1/knows>,
    <http://example.org/nmg>) IN foo

PRINT result
```

NOT MATCH reads exactly the way you would say it in English, which we were happy with.

4. Built-in operations

There are no user-defined functions in the language. Everything is a built-in.

LOAD reads a Turtle file. The file path is whatever name you pass with `.ttl` tacked on, looked for in the current directory. We did not write the Turtle parser ourselves; we used the `rdflib` library [7], which handles all the abbreviations the spec mentions (`@base`, `@prefix`, predicate lists, object lists). The triples it returns are converted into our own representation, which is a 3-tuple of `RDFNode` values, where `RDFNode` is `URI String | Str String | Num Int`.

UNION, **INTERSECT**, **MINUS** work on whole graphs and remove duplicates from the result. **UNION** is needed for Task 1 (and used in lots of the others); **MINUS** is needed for Task 6. We added **INTERSECT** for completeness even though no specific task requires it, since once you have two of the three set operators not implementing the third feels weird.

SELECT evaluates the **WHERE** clause to produce a list of variable bindings, then for each binding builds an output triple by substituting in the values. Duplicates are removed.

MATCH is the main pattern operator. It walks the graph, tries each triple, and accumulates bindings. When two **MATCH**es share a variable through an **AND**, the second one is fed the bindings of the first, which gives you a relational join. The implementation in `evalCond` just does this by chaining: `evalCond env (evalCond env bs c1) c2`.

Comparisons filter bindings. The numeric comparisons (`>=`, `<=`, `>`, `<`) require an integer-bound variable on the left and an integer literal on the right. `=` and `!=` work on any kind of value but only between values of the same kind.

Aggregates (MAX, MIN, SUM, COUNT) only do anything useful with GROUP BY. The interpreter splits the bindings into groups based on the value of the grouping variable and applies the aggregate to each group. MAX and MIN skip non-numeric values; COUNT just counts the bindings in the group; SUM adds them up.

PRINT outputs a graph as canonical N-Triples, sorted lexicographically on the rendered subject, then predicate, then object. We sort on the rendered string form (with the surrounding quotes for strings and angle brackets for URIs) because that automatically gives the ordering the spec asks for: in ASCII, " sorts before digits, which sort before <, so strings come before integers, which come before URIs.

5. Extensions

Beyond what was strictly required to solve the ten tasks, the language has several features we added either because they made the task programs cleaner or because they were obvious things that should be there.

5.1 Set operators as language primitives

Strictly speaking, you can do union with two SELECTs and an OR, and difference is harder but doable with NOT MATCH. Lifting UNION, INTERSECT and MINUS to the language level made several task programs much shorter. Task 6 is three loads and two operators, end of story. Task 7 would have been awful without MINUS.

5.2 Aggregates and GROUP BY

Task 4 only needs MAX. We implemented all four common aggregates (MAX, MIN, SUM, COUNT) because once we had the grouping infrastructure in place, the others were a few extra lines each, and the Part 2 spec hinted that the unseen tasks would be variations on the original ones.

5.3 Boolean conditions

AND, OR and NOT with the usual precedence (NOT tightest, then AND, then OR).

Implementation-wise, AND chains the two condition evaluations together (this is what makes shared variables behave like a join), OR evaluates each side independently and unions the result, and NOT keeps a binding only if no extension of it satisfies the inner condition. Parentheses can override precedence.

5.4 Comments

Anything from -- to the end of the line is ignored. This is one extra Alex rule. We added it pretty late but Task 7 in particular is long enough that being able to annotate what each intermediate graph represents makes the program a lot easier to read.

5.5 Type checking, sort of

The language is not statically typed and we did not write a type checker. The interpreter does keep track of whether each value is a URI, a string or an integer though, and the comparison operators only succeed when the types make sense: ?n >= 21 is only true if ?n is bound to a number. If it is bound to a string, the comparison returns false (rather than crashing or coercing). This is enough to keep ill-typed comparisons from corrupting the output, without us having to build a full type system.

5.6 Error handling

Three things can go wrong, and we tried to give a useful message for each:

- *Parse errors.* Happy's default error message is just `Happy: Parse error`, which is unhelpful. We added a custom `parseError` function that reports the offending token (or says "unexpected end of input" if it ran out).
- *Undefined names.* If you say `PRINT result` but never defined `result`, or you `UNION` two graphs and one of them doesn't exist, the interpreter says "`<name> is undefined`". The set operations check both operands separately so the message tells you which one is the problem.
- *Unbound output variables.* If a `SELECT`'s output list mentions a variable that doesn't appear in the `WHERE` clause (usually a typo), `buildTriple` reports "Unbound variable: `<name>`".

5.7 Syntax highlighting in VS Code

We built a VS Code extension (`rql-0.1.0.vsix`) for syntax highlighting. It's a `TextMate` grammar [6] that recognises seven token classes and assigns each one a standard scope name, so any colour theme will pick them up:

Class	TextMate scope	Examples
Comments	<code>comment.line.double-dash.rql</code>	<code>-- like this</code>
Strings	<code>string.quoted.double.rql</code>	<code>"Fred"</code>
URIs	<code>entity.name.type.rql</code>	<code><http://example.org/uos></code>
Integers	<code>constant.numeric.rql</code>	<code>21</code>
Variables	<code>variable.other.rql</code>	<code>?s, ?price</code>
Control keywords	<code>keyword.control.rql</code>	<code>SELECT, WHERE, MATCH, IN</code>
Set / logical / aggregate	<code>keyword.operator.*.rql</code>	<code>UNION, AND, MAX</code>

There is also a `language-configuration.json` that tells VS Code about the comment marker (`--`) and auto-closing pairs for parentheses, double quotes, and angle brackets. We packaged it with `vsce` and it installs into a normal VS Code build with no fuss.

6. Testing

We tested by writing small Turtle files for each task, running the corresponding `.rql` program, and checking the output against what we expected by hand. For each task we tried to put the input together so that it had at least the obvious case the task is asking for, plus a few extra triples that should *not* appear in the output, to make sure the filtering was actually doing something. For Task 2 (the age ≥ 21 one) we included someone whose age was exactly 21 and someone whose age was 20, to check the boundary.

Between them, the ten tasks cover most of the language. Tasks 1 and 6 use the set operations. Tasks 2 and 3 use basic pattern matching with constants and numeric comparisons. Task 4 uses aggregation. Task 5 uses a join across two graphs. Task 7 is the disjunction-by-difference one. Task 8 uses `NOT MATCH`. Task 9 uses self-joins on one graph. Task 10 rewrites predicates by putting different URIs in the output list. We are fairly confident every grammar production gets exercised by at least one task.

We also wrote a couple of broken programs to make sure the error messages came out properly: a program that uses an undefined graph name, one with a missing parenthesis in a `SELECT`, and one with a `SELECT` that mentions a variable not in the `WHERE`. All three printed the message we wanted instead of crashing.

If we had more time we would have set up `QuickCheck` to generate random graphs and check things like `g UNION g == g` automatically. We did not get round to it.

7. References

- [1] Harris, S. and Seaborne, A. (eds). *SPARQL 1.1 Query Language*. W3C Recommendation, 21 March 2013. <https://www.w3.org/TR/sparql11-query/>
- [2] Beckett, D., Berners-Lee, T., Prud'hommeaux, E. and Carothers, G. (eds). *RDF 1.1 Turtle - Terse RDF Triple Language*. W3C Recommendation, 25 February 2014. <https://www.w3.org/TR/turtle/>
- [3] Schreiber, G. and Raimond, Y. (eds). *RDF 1.1 Primer*. W3C Working Group Note, 24 June 2014. <https://www.w3.org/TR/rdf11-primer/>
- [4] Marlow, S. and Gill, A. *Happy: The Parser Generator for Haskell*. <https://www.haskell.org/happy/>
- [5] Dornan, C. and Marlow, S. *Alex: A Lexical Analyser Generator for Haskell*. <https://www.haskell.org/alex/>
- [6] Microsoft. *Syntax Highlight Guide*. Visual Studio Code Documentation. <https://code.visualstudio.com/api/language-extensions/syntax-highlight-guide>
- [7] *rdf4h: An RDF library for Haskell*. Hackage. <https://hackage.haskell.org/package/rdf4h>
- [8] Date, C. J. *SQL and Relational Theory*, 3rd edition. O'Reilly, 2015.